

Prathiksha Patil
261100690008

Day-2: File Ownership and Access Control

Topics: chown, chgrp, ownership + permissions

Part A — Ownership Investigation

1. Create a directory named ownership_lab.

```
mkdir ownership_lab
```

2. Inside it, create: incident_report.txt, network.log, credentials.txt

```
cd ownership_lab
```

```
touch incident_report.txt network.log credentials.txt
```

3. Display the detailed information for these files.

```
bash
```

```
ls -l
```

4. For each file, identify Owner, Group, Owner permissions, Group permissions, Others permissions.

All three files are identical:

- Owner: prathiksha
- Group: prathiksha
- Owner permissions: rw- (Read & Write)
- Group permissions: rw- (Read & Write)
- Others permissions: r-- (Read-only)

5. Determine the UID and GID of the current user.

```
id
```

6. Compare the UID/GID information with the ownership displayed for the newly created files.

The comparison shows an exact match. All three files are owned by user prathiksha and group prathiksha. The `id` command confirms the active terminal session is running under the identity of prathiksha (UID 1000) and group prathiksha (GID 1000). In Linux, any newly created file automatically inherits the UID of the creator as the Owner, and the creator's primary GID as the Group owner — this is why the numbers line up exactly with no manual `chown/chgrp` needed.

7. Explain the difference between file ownership and file permissions.

- File Ownership answers the question "Who?" — it designates a specific user account (Owner) and a specific group (Group) as the file's assigned holders.
- File Permissions answer the question "What?" — they define the concrete actions (Read, Write, Execute) allowed for the owner, the group members, and everyone else (others) on the system.

```
(prathiksha@kali)~[~]
$ mkdir ownership_lab
network.log
(prathiksha@kali)~[~]
$ cd ownership_lab
ownership_lab
credentials.txt
(prathiksha@kali)~/ownership_lab
$ touch incident_report.txt network.log credentials.txt
(prathiksha@kali)~/ownership_lab
$ ls -l
total 0
-rw-rw-r-- 1 prathiksha prathiksha 0 Sep  3 22:56 credentials.txt
-rw-rw-r-- 1 prathiksha prathiksha 0 Sep  3 22:56 incident_report.txt
-rw-rw-r-- 1 prathiksha prathiksha 0 Sep  3 22:56 network.log
(prathiksha@kali)~/ownership_lab
$ id
uid=1000(prathiksha) gid=1000(prathiksha) groups=1000(prathiksha),4(adm,lpadmin)
```

Part B — chown: Changing Ownership

1. Change the owner of incident_report.txt to alice.

```
sudo chown alice incident_report.txt
```

2. Change the owner of network.log to bob.

```
sudo chown bob network.log
```

3. Change the owner of credentials.txt to charlie.

```
sudo chown charlie credentials.txt
```

4. Verify each change.

```
ls -l
```

5. Try changing the owner of a file to another user without using sudo. Record the result.

```
chown bob incident_report.txt
```

6. Explain why an ordinary user is generally not permitted to transfer file ownership to another user.

- Quota evasion: If regular users could freely give away files, someone running low on their disk quota could transfer large files to another user's ownership, dumping the storage burden onto someone else without their consent.
- Security & accountability: An unprivileged user could create a malicious script, make it executable, and then hand ownership to root or another privileged user — hoping it gets executed under that identity, which would be a serious privilege-escalation vector. Restricting chown to root (or via sudo) keeps ownership transfers controlled and auditable, maintaining a clear chain of accountability for every file on the system.

7. Using a single command, configure incident_report.txt with Owner: alice, Group: security.

```
sudo chown alice:security incident_report.txt
```

```
(prathiksha@kali) [~/ownership_lab]
$ sudo useradd -m alice
sudo useradd -m bob
sudo useradd -m charlie
useradd: user 'alice' already exists
useradd: user 'bob' already exists
useradd: user 'charlie' already exists

(prathiksha@kali) [~/ownership_lab]
$ sudo chown alice incident_report.txt

(prathiksha@kali) [~/ownership_lab]
$ sudo chown bob network.log

(prathiksha@kali) [~/ownership_lab]
$ sudo chown charlie credentials.txt

(prathiksha@kali) [~/ownership_lab]
$ ls -l
total 0
-rw-rw-r-- 1 charlie prathiksha 0 Sep  3 22:56 credentials.txt
-rw-rw-r-- 1 alice   prathiksha 0 Sep  3 22:56 incident_report.txt
-rw-rw-r-- 1 bob     prathiksha 0 Sep  3 22:56 network.log

(prathiksha@kali) [~/ownership_lab]
$ chown bob incident_report.txt
chown: changing ownership of 'incident_report.txt': Operation not permitted

(prathiksha@kali) [~/ownership_lab]
$ sudo chown alice:security incident_report.txt

(prathiksha@kali) [~/ownership_lab]
$ ls -l incident_report.txt
-rw-rw-r-- 1 alice security 0 Sep  3 22:56 incident_report.txt
```

Part C — chgrp: Changing Group Ownership

1. Change the group ownership of network.log to developers.

```
sudo chgrp developers network.log
```

2. Change the group ownership of credentials.txt to security.

```
sudo chgrp security credentials.txt
```

3. Verify the changes.

```
ls -l
```

4. Display the group memberships of alice, bob, and charlie.
groups alice groups bob groups charlie

5. Log in as alice and create alice_report.txt.
sudo su - alice cd ownership_lab touch alice_report.txt

6. Attempt to change the group of alice_report.txt to security.
chgrp security alice_report.txt

7. Attempt to change its group to developers.
chgrp developers alice_report.txt

8. Compare the results.
chgrp: changing group of 'alice_report.txt': Operation not permitted

9. Explain how a user's group membership affects their ability to change the group ownership of a file they own. A regular user can change the group of a file they own only to a group they are already a member of. So, their group membership decides which groups they can assign without using admin privileges.

```
(prathiksha@kali)~/ownership_lab
$ sudo chgrp developers network.log

(prathiksha@kali)~/ownership_lab
$ sudo chgrp security credentials.txt

(prathiksha@kali)~/ownership_lab
$ ls -l
total 0
-rw-rw-r-- 1 charlie security 0 Sep 3 22:56 credentials.txt
-rw-rw-r-- 1 alice security 0 Sep 3 22:56 incident_report.txt
-rw-rw-r-- 1 bob developers 0 Sep 3 22:56 network.log

(prathiksha@kali)~/ownership_lab
$ groups alice
alice : alice

(prathiksha@kali)~/ownership_lab
$ groups bob
bob : bob security developers

(prathiksha@kali)~/ownership_lab
$ groups charlie
charlie : charlie security

(prathiksha@kali)~/ownership_lab
$ sudo su - alice
$ cd ownership_lab
-sh: 1: cd: can't cd to ownership_lab
$ touch alice_report.txt
$ chgrp security alice_report.txt
chgrp: changing group of 'alice_report.txt': Operation not permitted
$ chgrp developers alice_report.txt
chgrp: changing group of 'alice_report.txt': Operation not permitted
$
```

Part D — Ownership + Permission Investigation

1. Configure -rw-r----- alice security incident_report.txt
sudo chmod 640 incident_report.txt

2. Identify what operations are available to: alice, charlie, bob
sudo usermod -aG security Charlie

3. A user who belongs to neither security nor the file owner charlie belongs to security. Record the result of each operation. Test whether Charlie can:

- Read incident_report.txt
- Modify incident_report.txt
- Delete incident_report.txt

sudo chmod 660 incident_report.txt

sudo su - charlie

cd ownership_lab

cat incident_report.txt

echo "test" >> incident_report.txt

rm incident_report.txt

4. Explain why Charlie's access changed even though the owner and group of the file did not change. Remove bob from the security group.

Charlie's access changed because the group permissions were changed from read-only (r--) to read and write (rw-). The group remained security, so Charlie, as a member of that group, automatically got write access
sudo gpasswd -d bob security

5. Explain why Bob's effective permissions changed.

Linux checks file permissions in this order:

User → Group → Others.

When Bob was in the security group, he got the group permissions (rw-). After Bob was removed from the group, he no longer had group access, so Linux used the Others permissions (---), which means he had no access.

```
(prathiksha@kali)-[~/ownership_lab]
$ sudo chmod 640 incident_report.txt

(prathiksha@kali)-[~/ownership_lab]
$ sudo usermod -aG security charlie

(prathiksha@kali)-[~/ownership_lab]
$ sudo chmod 660 incident_report.txt

(prathiksha@kali)-[~/ownership_lab]
$ sudo su - charlie
$ cd /home/prathiksha/ownership_lab

$ cat incident_report.txt
$ sudo chgrp security incident_report.txt
[sudo] password for charlie:
charlie is not in the sudoers file.
$ echo "test" >> incident_report.txt
$ rm incident_report.txt
$ exit
```

```
(prathiksha@kali)-[~/ownership_lab]
$ sudo gpasswd -d bob security
Removing user bob from group security

(prathiksha@kali)-[~/ownership_lab]
$ sudo su - bob
```